

React Day 2 Notes

Project Folder Structure

```
None
src
|
├── components
|   ├── Navbar.jsx
|   ├── Section.jsx
|   ├── Doctorcard.jsx
|   └── styles.css
|
├── App.jsx
└── main.jsx
```

React Code- Day 2

Part 1: Creating Multiple Cards Using Same Component + Flexbox

Concept

A component can be reused multiple times to display the same UI structure.

Doctorcard.jsx

```
None
function Doctorcard() {
```

```

return (
  <div className="card">
    <div>
      
    </div>

    <h2>JOHN</h2>
    <p>Muscles Specialist</p>
    <p>Male</p>

    <button>View Details</button>
  </div>
);
}

export default Doctorcard;

```

App.jsx

```

None
import Doctorcard from "../components/Doctorcard";
import Navbar from "../components/Navbar";
import Section from "../components/Section";

```

```
import "../components/styles.css";

function App() {
  return (
    <div>
      <Navbar />
      <Section />

      <div className="doctorparent">
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
      </div>
    </div>
  );
}

export default App;
```

styles.css

```
CSS
.card {
```

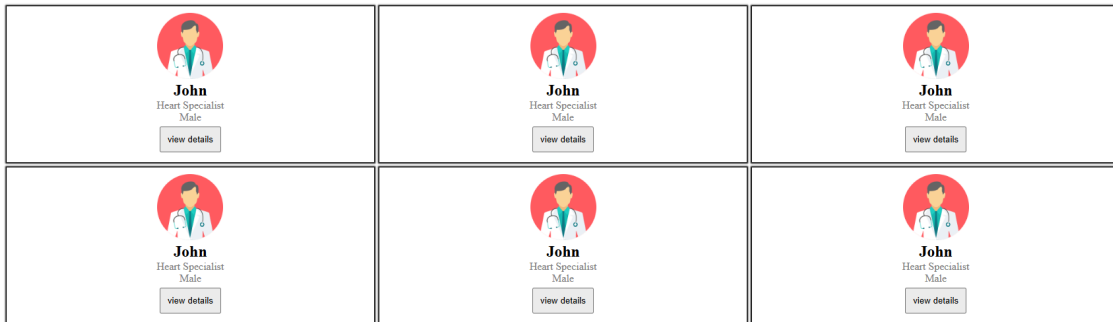
```
    box-shadow: 0px 0px 2px 2px black;
    padding: 10px;
    text-align: center;
}
```

```
.card > p {
    font-weight: 300;
    color: grey;
}
```

```
.doctorparent {
    display: flex;
    flex-wrap: wrap;
    gap: 8px;
    justify-content: center;
    margin: 20px;
}
```

```
.doctorparent > div {
    flex-basis: 30%;
}
```

Part-1 Output



Part 2: Dynamic Cards Using Props

Technical Definition of Props

Props (Properties) are read-only inputs passed from a parent component to a child component for transferring dynamic data.

App.jsx

None

```
import Doctorcard from "../components/Doctorcard";
import Navbar from "../components/Navbar";
import Section from "../components/Section";
import "../components/styles.css";

function App() {
  return (
    <div>
      <Navbar />
      <Section />

      <div className="doctorparent">
```

```
<Doctorcard
  name="Sivaram"
  specialization="Bones"
  gender="male"
/>
```

```
<Doctorcard
  name="Teja"
  specialization="Muscles"
  gender="male"
/>
```

```
<Doctorcard
  name="Vasavi"
  specialization="Psycho"
  gender="female"
/>
```

```
<Doctorcard
  name="Madhu"
  specialization="Orthocare"
  gender="male"
/>
```

```
<Doctorcard
  name="Aditya"
```

```

        specialization="Skin Specialist"
        gender="male"
    />

    <Doctorcard
        name="John"
        specialization="Lungs"
        gender="male"
    />
</div>
</div>
);
}

export default App;

```

Receiving Props Using props Object

Doctorcard.jsx

```

None
function Doctorcard(props) {
    return (
        <div className="card">
            <div>
                
    </div>

    <h2>{props.name}</h2>
    <p>{props.specialization}</p>
    <p>{props.gender}</p>

    <button>View Details</button>
  </div>
);
}

export default Doctorcard;
```

Deconstructing Props

Technical Definition

Deconstructing is a JavaScript feature used to extract values from objects or arrays and store them directly in variables.

Doctorcard.jsx

None

```
function Doctorcard({ name, specialization, gender }) {
  return (
    <div className="card">
```

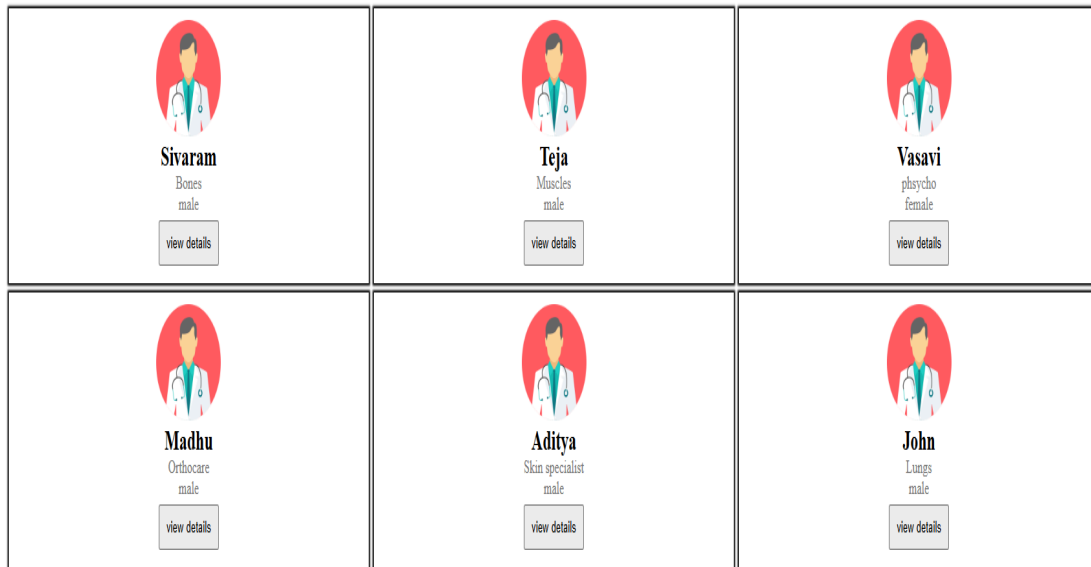
```
        <div>
          
      </div>

      <h2>{name}</h2>
      <p>{specialization}</p>
      <p>{gender}</p>

      <button>View Details</button>
    </div>
  );
}

export default Doctorcard;
```

Part-2 Output



Summary

Part 1

- Created reusable DoctorCard component
- Called same component multiple times
- Used Flexbox for layout
- Learned component reusability

Part 2

- Passed dynamic data using Props
- Received data using **props**
- Used Props Destructuring
- Built dynamic and reusable cards